

INO Security Audit

Source-level review of the INO personal wealth and document vault before Play Store submission. Every finding below was read in the code, and each Critical and High item was independently re-verified by a second pass. Suspected issues that turned out to be safe are listed separately so they are not mistaken for bugs.

Audited 3 Sep 2026 Branch main at 07b1acd plus working-tree changes

351 Dart files · 30 migrations · 3 edge functions · Next.js share site · Android/iOS config

OVERALL SCORE

4/10

Not ready to submit

Strong foundations undermined by one remotely exploitable hole, two bypassable client gates, and four Play Console blockers.

Backend & RLS		7
Platform & build		6
Auth & session		5
Crypto & local data		5
Sharing & deep links		3
Privacy & Play policy		3

4 Critical

9 High

17 Medium

17 Low

7 Info

28 controls verified sound

How to read the score

The data model, cryptography and auth plumbing are better than most apps at this stage. Row-level security is on every table the app uses, the password vault is genuinely end-to-end encrypted with AES-256-GCM and a 210k-round PBKDF2 key, sign-in uses PKCE, no real secret has ever been committed, and there are no trackers or ad SDKs. That is why the backend area scores 7.

The score is pulled to 4 because the surfaces users and reviewers actually touch have real failures. Anyone on the internet can enumerate and download every active share without a link. Two-factor authentication and the app lock can both be stepped around on-device. The secrets the vault exists to protect are re-written as

plaintext into backup-eligible preferences. And four separate items will stop a Play Console submission outright before any of that matters.

Launch blockers

These four must be fixed before the app can be submitted or safely used by the public.

C1 Anyone can enumerate and download every active share without its link

CRITICAL re-verified

supabase/functions/share/index.ts:157 • deep_link_service.dart:156-176

The share edge function drops the raw URL path segment straight into a PostgREST `.or()` filter, runs it with the service-role client, and is deployed without JWT verification and with open CORS. Path characters like `,` and `(` are legal, so a caller injects extra filter clauses. The three response classes (not found, served, or a multi-row error) form a bisection oracle: roughly 50 requests per share recovers each 12-character token, after which the same path serves the document bytes. There is no rate limiting, and the world-readable wallets registry leaks user UUIDs to target a specific victim's shares.

Fix Validate the segment against `^(share_)?[0-9a-f]{12,18}$` and return 404 otherwise. Replace the string-built `.or()` with two parameterised `.eq()` lookups. Add per-IP rate limiting. Apply the same allow-list in the app's deep-link parser as defence in depth.

C2 "Delete account" does not delete the account **CRITICAL** re-verified

lib/services/account_service.dart:89-139 • no delete_account in supabase/

The client removes storage files, wallet rows and the profile row, then calls an RPC named `delete_account` inside a try/catch that logs and ignores failure. That RPC is not defined anywhere in the repository. The auth user survives, so the person can sign straight back in, and none of the cascade deletes fire. Notes, reminders, expenses, tax documents, family-vault membership, shares (which keep serving), push logs and payment QR rows remain. The in-app copy promises permanent removal. Play's account-deletion policy and India's DPDP Act both require actual deletion.

Fix Ship a `security definer delete_account()` that removes storage objects and then deletes from `auth.users` so the 27 cascading foreign keys do the rest. Make the client treat RPC failure as a hard error. Commit the function to a migration.

C3

No hosted privacy policy URL and no web account-deletion URL **CRITICAL**`lib/screens/legal/legal_document_screen.dart` • `README.md`

The policy exists only as bundled in-app strings. Play Console requires a public URL for any app using camera, microphone or biometrics, plus a web page for deletion requests when the app offers account creation. Neither exists.

Fix Host both on the existing Vercel share site, link them from the legal screen, and enter the URLs in Play Console. Rewrite the policy first (see H8).

C4

Application ID is still `com.example.inoapp` **PLAY BLOCKER** **re-verified**`android/app/build.gradle.kts:25,45` • `firebase_options.dart` • `assetlinks.json` • `google-services.json`

Play refuses any `com.example.*` package. The ID is immutable after first upload, and the Firebase Android app, Google OAuth client, and asset links are all bound to it, so the rename cascades.

Fix Rename to a real reverse-domain ID, move MainActivity to the matching package, re-run `flutterfire configure`, re-register SHA fingerprints on the OAuth client, update assetlinks.

High severity

H1

Two-factor authentication is bypassed by force-quitting at the TOTP prompt**HIGH** **re-verified**`lib/screens/auth/auth_flow.dart:57-71` • `lib/screens/splash/splash_screen.dart:257-260`

Password sign-in persists an aal1 session before the MFA screen is pushed. The MFA check lives only in the interactive sign-in flow. On cold start the splash screen sees a session and opens the shell with no AAL check, and no RLS policy checks `auth.jwt()` -> `'aal1'`. An attacker with the password swipes the app away at the prompt and reopens it. The account switcher also saves the aal1 refresh token.

Fix Call `needsMfaChallenge()` in the splash path and on account switch. Add an aal2 condition to RLS policies on sensitive tables so an aal1 token cannot read data even if the client is bypassed. Do not save the account to the switcher until aal2.

H2

Logged-out phone in airplane mode opens the previous user's offline documents **HIGH** re-verified

`splash_screen.dart:250-255` · `auth_service.dart:253` · `session_reset.dart` (no `OfflineDocumentStore`)

When offline, the splash routes to the offline library before checking for a session. Sign-out turns the biometric app lock off, wipes the per-document protection flags, and never purges the offline files or the "last uid" pointer. Aadhaar and passport scans saved offline stay readable on a shared or resold device.

Fix Add an offline-store purge to `SessionReset`, or refuse the offline root when no session exists and require device credential before rendering it. Do not disable the app lock on sign-out while offline data remains.

H3

Password vault entries cached as plaintext JSON in SharedPreferences **HIGH**

`lib/services/password_store.dart:36-51` · `local_collection_store.dart:243-254`

The server copy is AES-GCM sealed and the key lives only in memory, but every decrypted entry is written back to `ino_passwords_<uid>` in preferences. The passphrase gate is a UI gate; the plaintext is on disk regardless. The file's own comment acknowledges this. Root, forensic extraction, or a backup restore yields every password with no cryptographic work.

Fix Persist the sealed form and decrypt lazily while unlocked, or move the cache to `flutter_secure_storage`, which is already a dependency.

H4

Refresh tokens for every saved account "protected" by a constant single-byte XOR **HIGH**

`lib/services/account_switcher.dart:34-53,241-251`

The comment says "XOR cipher with device salt". There is no salt; it is XOR with `0x57` then base64. A refresh token is a long-lived bearer credential usable from any machine. Every account ever signed in on the device is kept, and removing one does not revoke it server-side. The live Supabase session also sits in plain `SharedPreferences` because `Supabase.initialize` is not given a secure `localStorage`.

Fix Store tokens in `flutter_secure_storage`, pass a secure-storage-backed `localStorage` to Supabase, and best-effort revoke on remove.

H5

Android auto-backup left on; secrets and scans are backup and device-transfer eligible HIGH re-verified

`android/app/src/main/AndroidManifest.xml` (no `allowBackup`, no `dataExtractionRules`, no `res/xml/`)

With the default, Android backs up `shared_prefs/` and `files/`. That set includes the plaintext passwords (H3), the XOR'd tokens (H4), the live session, the document metadata cache with ID numbers, the profile cache, and the unencrypted offline scans. Device-to-device transfer copies them regardless of quota. This one setting compounds three other High findings.

Fix Add `android:allowBackup="false"`, or ship `dataExtractionRules` plus `fullBackupContent` excluding preferences and the offline directory.

H6

Unauthenticated callers can run schema-changing RPCs HIGH re-verified

`supabase/migrations/20260727000000_per_wallet_tables.sql:112-226`

`ino_register_wallet` and `ino_create_wallet_table` are SECURITY DEFINER functions that create tables, policies and triggers. Neither checks `auth.uid()`, and no grant or revoke exists for either anywhere in the repo, so Supabase's default EXECUTE grant leaves them callable with the anon key. The sign-in check and the 200-wallet cap live only in the wrapper `create_custom_wallet`. Names are slug-sanitised, so this is unbounded DDL rather than SQL injection.

Fix revoke all on function `public.ino_register_wallet(...)` from `public`, `anon`, `authenticated`; and the same for `ino_create_wallet_table`. Do the same for `expire_due_shares()` and `ino_rebuild_documents_view()`.

H7

Full Aadhaar, PAN and passport numbers stored plaintext server-side and shown unmasked HIGH

`lib/services/aadhaar_parser.dart:82-103` • `add_document_screen.dart:158-230` • `models/document.dart:66-74` • `document_viewer_screen.dart:927,971`

OCR yields the full 12-digit Aadhaar plus name, date of birth and gender, all written to `record_number` and a JSON envelope in `notes`, uploaded with no client-side encryption. No masking exists anywhere. The values also land in the JSON cloud backups, the local metadata cache, and the "military-grade encryption" onboarding copy is shown while none of this is client-encrypted. UIDAI guidance discourages storing full Aadhaar at all.

Fix Display-mask by default (`xxxx xxxx 1234`). Encrypt `record_number` and `notes` with the existing vault key pattern, or store only the last four digits plus a hash.

H8

Privacy policy is thin, omits every third party, and makes a false export claim

HIGH

`lib/l10n/app_localizations.dart:2338-2348`

Six short paragraphs. Names no processor (Supabase, Firebase Cloud Messaging, ML Kit, Google Sign-In, Google Fonts, Swissquote, Frankfurter, Vercel, the OS speech recogniser), no retention period, no age policy, no DPDP or GDPR rights, no legal entity, and gives no direct contact. It states users can export all their data at any time. No export UI exists (M15).

Fix Rewrite against the Play Data Safety vocabulary and DPDP Act 2023, using the data-safety map in this report as the source of truth. Keep the in-app text and hosted page in sync.

H9

A custom debug keystore is committed and signs the "release" APK that CI publishes

HIGH

re-verified

`android/app/debug.keystore` • `.gitignore:57` (explicit un-ignore) • `build.gradle.kts:84-96` • `.github/workflows/build-apk.yml:40,69`

The keystore is not the stock SDK key; it is a team key created in August 2026, deliberately un-ignored. When `key.properties` is absent the release build type falls back to it with only a warning, and the CI workflow runs `flutter build apk --release` with no signing secrets and publishes the result as the latest GitHub release. Anyone with repo access can sign an APK Android accepts as an update to every tester's install, inheriting their data directory. Play itself will reject a debug-signed bundle, so production is not directly at risk; the tester channel is. The release keystore has never been committed, which is correct.

Fix Rotate and stop tracking the keystore, or feed a real upload key into CI from secrets and make the release build fail rather than warn when `key.properties` is missing.

Medium severity

M1

A vault admin can mint an owner MEDIUM re-verified

supabase/migrations/20260728000000_family_vaults.sql:297-298

The `vault_members` INSERT policy checks only `is_vault_admin`. UPDATE and DELETE guard `role <> 'owner'`; INSERT does not. An admin inserts a row with `role='owner'` directly via REST, bypassing the owner-only promotion RPC, and can add any UUID as a member with no invitation.

Fix Add `and role in ('admin','editor','viewer')` to the INSERT policy's `with check`.

M2

User-directory enumeration through vault invites MEDIUM

supabase/migrations/20260901010000_family_vault_join_and_cowners.sql:199-265

`invite_ino_user_to_vault` resolves any email, phone or name against the whole users table and returns distinct not-found and ambiguous errors. Any user can create a vault to become admin, then probe whether a given person uses INO.

Fix Return a uniform response, rate-limit per caller.

M3

Sender-controlled MIME type is served inline on the share origin MEDIUM

share_repository.dart:204-206 • 20260810_share_security_fixes.sql:39-93 • functions/share/index.ts:466-474 • share-frontend route.ts:29-31

`create_processed_share` accepts any MIME. A malicious sender uploads HTML or SVG as a processed copy and sends the link; it renders as a page on the INO share domain with no `nosniff` or CSP, a ready-made phishing page.

Fix Allow-list MIME on upload, derive Content-Type server-side, add `X-Content-Type-Options: nosniff` and `Content-Security-Policy: sandbox` on file responses.

M4

Share password hash is the credential and travels in the query string MEDIUM

lib/utils/share_password.dart:6-11 • index.ts:224-232,948-956 • ShareUnlock.tsx:36-38

Passwords are hashed with unsalted SHA-256, the server accepts the hash as-is, and both browser and app send it as `?pw=`. It lands in Vercel and Supabase logs, browser history, and Referrer headers. A database read yields a directly replayable credential.

Fix Move to a POST body, hash with `pgcrypto` `crypt()` server-side, never accept the hash, add `Referrer-Policy: no-referrer`.

M5

No rate limiting on the public share endpoint MEDIUM`functions/share/index.ts:283-308`

Password attempts and view inserts are unthrottled. Password-protected shares can be brute-forced at wire speed and analytics tables grown without bound.

Fix Per-IP and per-token counters on password attempts and views.

M6

The 5 GB storage quota is enforced only in the client MEDIUM`document_repository.dart:135-141 · storage_and_quota_setup.sql (read-only usage function)`

A direct Storage API call with the user's own JWT ignores the cap.

Fix Enforce with a `storage.objects` INSERT policy or trigger that sums the user's folder.

M7

Offline documents and every viewed document sit unencrypted on disk and are never purged MEDIUM`offline_document_store.dart:298-332 · document_file_service.dart:15-47`

Every viewed document is streamed to the temp directory and kept. Offline copies are plain file copies. Sign-out clears no files. The biometric "protected" flag gates the UI only. Storage is app-private, which is correct, but it is in the backup set (H5).

Fix Delete the temp document cache on sign-out and after open; encrypt offline copies with a Keystore-wrapped key; exclude from backup.

M8

Document metadata cache with ID numbers and profile cache in preferences outlive sign-out MEDIUM`document_repository.dart:93-121 · user_repository.dart:113-125, 277-288`

Uid-scoped so no cross-account leak, but the full `record_number` values, notes, name, email and phone persist in plain preferences after sign-out.

Fix Remove on sign-out; encrypt or exclude from backup.

M9

Media permissions will trigger Play's Photo and Video policy review

PLAY POLICY

`AndroidManifest.xml:30 · manifest merger adds READ_MEDIA_VIDEO and READ_MEDIA_AUDIO from open_filex`

Play requires apps with one-off gallery access to use the Photo Picker and not declare `READ_MEDIA_IMAGES`. The image picker already uses the Photo Picker on API 33+, so the permission adds nothing. Video and audio are unused.

Fix Remove the declaration and add `tools:node="remove"` entries for all three.

M10

Notification bodies carry document names and card details to the lock screen and to an unpruned log **MEDIUM**

functions/send-push/index.ts:190-306 · push_service.dart:82-92 · 20260813_push_log.sql

"Aadhaar Card expires today" is readable on a locked phone because `NotificationVisibility` is never set. Titles and bodies are stored in `push_log` and `notification_outbox` indefinitely.

Fix Use `NotificationVisibility.private` with a generic public version, keep details in `data`, add retention.

M11

Voice navigation uses the cloud speech recogniser by default and logs transcripts **MEDIUM**

voice_navigation_service.dart:86-93,198-203

`preferOffline` defaults to false, so audio may go to Google or Apple. No in-app rationale precedes the microphone prompt. This is "voice recordings shared" on the Data Safety form and the policy says nothing about it.

Fix Disclose, prefer on-device where available, drop transcript logging.

M12

Consent gaps: push prompt at launch, cosmetic Notifications toggle, Google sign-in skips terms **MEDIUM** re-verified

main.dart:133 · profile_screen.dart:77,220-226 · app_settings.dart · login_screen.dart

The FCM permission prompt fires one frame after launch for everyone including guests. The Profile "Notifications" switch persists a boolean that nothing reads except the export. Email sign-up requires the terms checkbox; "Continue with Google" creates an account without it.

Fix Defer the push prompt to first reminder; honour the toggle by deleting the device token; show terms consent on first Google sign-in.

M13

No age gate or 18+ statement for a financial-data app **PLAY POLICY**

Terms strings in `app_localizations.dart`; no age reference anywhere in `lib/`

Play needs a target-audience declaration; DPDP requires parental consent for under-18s. An explicit 18+ clause is the cheap path.

Fix Add the clause to Terms and sign-up, declare 18+ in Play Console.

M14

Passwords and ID numbers copied to clipboard without auto-clear **MEDIUM**

password_vault_screen.dart:223-230 · document_viewer_screen.dart:1741 · module_kit.dart:177 (contrast vault_item_detail_screen.dart:26-42 which clears after 30 s)

Fix One `SecureClipboard.copy()` helper with a 30–60 s wipe, used everywhere.

M15

Data export and backup are advertised but unreachable **MEDIUM** re-verified

data_export_service.dart, backup_service.dart (only caller: auto_backup_coordinator.dart) · profile_screen.dart:780-840

No screen invokes the export, and the auto-backup toggle has no UI, so the default stays off. The policy, FAQ and Terms all describe it as available.

Fix Wire a Profile row to the existing service, or remove the claims.

M16

iOS Podfile compiles out the microphone and speech permission handlers**FUNCTIONAL**

ios/Podfile:56-60

`PERMISSION_MICROPHONE=0` and `PERMISSION_SPEECH_RECOGNIZER=0` mean voice navigation never prompts on iOS despite the usage strings being present.

Fix Set both to 1, or drop the feature and strings on iOS.

M17

The core per-user folder policy for the documents bucket is not in version control **VERIFY** re-verified

No `storage.objects` policy for the documents bucket in any migration; `storage_and_quota_setup.sql` is only a usage function

The single most important storage control, restricting reads and writes to the caller's own `<uid>/` folder, is only referenced in comments as "your existing Storage RLS". If it is missing or wrong in the dashboard, users can read each other's files. This could not be verified from the repo.

Fix Confirm in `Storage → Policies` that all four operations are scoped to `(storage.foldername(name))[1] = auth.uid()::text`, then commit it to a migration.

Low severity

ID

FINDING

LOCATION

FIX

L1	Screenshot protection only on the view-once viewer; document viewer, password vault and cards are capturable. App lock arms on <code>pause</code> and <code>not inactive</code> .	<code>screen_security_service.dart</code> · <code>app_lock.dart:82-98</code>	Enable the existing ref-counted <code>FLAG_SECURE</code> on those screens.
L2	"Trusted devices" is a local-only list that gates nothing; "Remove" edits a preference string. No sign-out-everywhere exists.	<code>trusted_device_service.dart:193-200</code>	Back it with a sessions table or relabel it "This device".
L3	Account switch requires no re-authentication.	<code>profile_screen.dart:616-635</code>	Gate with <code>VaultGuard.ensureUnlocked(force:true)</code> .
L4	Google Sign-In has no nonce (acknowledged in docs).	<code>auth_service.dart:234-240</code>	Pass a SHA-256 nonce to Google and the raw nonce to Supabase.
L5	Wallets registry is readable by every user and exposes <code>created_by</code> UUIDs and custom labels.	<code>20260727000000_per_wallet_tables.sql:96-100</code>	Expose through a view without <code>created_by</code> .
L6	<code>device_tokens</code> UPDATE policy uses <code>using (true)</code> ; knowing a token string lets a user re-stamp it.	<code>20260727120000_device_tokens.sql:65-66</code>	Scope to owner, delete-then-insert on switch.
L7	<code>expire_due_shares()</code> and <code>ino_rebuilt_documents_view()</code> callable by PUBLIC.	<code>20260704000000_document_shares.sql:176-189</code>	Revoke, grant to <code>service_role</code> .

L8	Legacy <code>share_schema.sql</code> (anon-callable resolver returning storage paths, PUBLIC bucket) and unused <code>vault_schema.sql</code> still in the tree.	<code>supabase/share_schema.sql:34-55</code>	Delete so nobody pastes them.
L9	Sender-controlled filename and MIME handed to <code>OpenFile</code> <code>x ; an .apk share</code> launches the installer (user still confirms).	<code>shared_documents_screen.dart:161-167</code>	Allow-list MIME before opening.
L10	Share site loads the <code>pdf.js</code> worker from <code>unpkg</code> and sets no security headers.	<code>share-frontend DocViewer.tsx:15</code> · <code>next.config.js</code>	Bundle the worker; add CSP, nosniff, frame-ancestors, Referrer-Policy.
L11	Deep-link and QR tokens accepted in any shape and string-concatenated into the backend URL; any <code>https://host/s/x</code> is treated as an INO share.	<code>deep_link_service.dart:156-176</code> · <code>payment_qr.dart:67-79</code> · <code>share_config.dart:38,63</code>	Regex-validate the token, check the host.
L12	App Links unverifiable: <code>assetlinks.json</code> has a placeholder fingerprint, iOS entitlements say <code>your-share-domain</code> , and the manifest host is the Supabase functions domain rather than the Vercel share domain.	<code>deep-linking/assetlinks.json:10</code> · <code>Runner.entitlements:23</code> · <code>AndroidManifest.xml:95-103</code>	Point at the Vercel host, publish the Play App Signing SHA-256.

L13	Release build has no Dart obfuscation.	build-apk.yml · RELEASE_SIGNING.md	--obfuscate - -split-debug-info .
L14	"MILITARY-GRADE AES ENCRYPTION" and "256-BIT AES ENCRYPTION ACTIVE" shown for data that is not client-side encrypted. Play's misleading-claims policy risk.	app_localizations.dart:1297,1845,2330	Say "private, access-controlled vault" unless documents become E2E encrypted.
L15	Vault key stays in memory until sign-out; not dropped on background despite the code comment claiming it is.	app_lock.dart:82-98 · vault_crypto.dart:219-224	Call VaultCrypto.lock() on pause.
L16	Emails, voice transcripts and the full push data map written to developer.log without a debug guard. Discarded in release AOT builds, so exposure is debug-only.	auth_service.dart:208 · voice_navigation_service.dart:202 · push_service.dart:408	Guard with kDebugMode .
L17	Offline documents saved with no resolved uid go under a _local key merged into every account's library.	offline_document_store.dart:180,227-233	Drop the fallback or scope it.

Informational

- **I1** A second, older password-vault stack (`VaultController` , `VaultKeyStore` , `screens/vault/`) is dead code, not imported anywhere outside its folder. It would cache the raw AES key in secure storage with no expiry if ever wired back in. Delete it.
- **I2** The app passes `p_password` and `p_is_view_only` to `create_document_share` , which only accepts two arguments in the repo. Password-protected shares of originals fail closed unless the live database was hand-patched.
- **I3** `ITSAAppUsesNonExemptEncryption` is false while the app implements AES-GCM. Standard algorithms are exempt but an annual self-classification may still apply.
- **I4** Firebase API keys in source are normal identifiers. Whether they are restricted to the package and SHA-256 cannot be verified from the repo. Analytics is enabled on the Firebase project but no analytics SDK ships in the app.
- **I5** CI pins Flutter 3.44.8 while the local toolchain is 3.38.5.
- **I6** Three tests in `data_isolation_test.dart` and `duplicate_username_isolation_test.dart` fail because `ReminderStore` now calls Supabase directly and the harness never initialises it. Reminder account-switch isolation is therefore untested. All other security-related suites pass (138 of 141).
- **I7** Non-Latin document names in `Content-Disposition` will make the share function return 500; use the `filename*=UTF-8''` form.

What was checked and found sound

These are the controls that hold. They are listed so nobody re-audits them and so the score is read in context.

- **Row-level security** is enabled on every table the app uses, with owner-scoped select, insert, update and delete policies. No `anon` policies, no grants to anon, both views use `security_invoker` .
- **SECURITY DEFINER functions** pin `search_path` and re-verify ownership or role inside the function body, including all share and view-once RPCs.
- **View-once links** are consumed by a single atomic UPDATE; concurrent claims cannot both succeed. The peek, claim and resolve RPCs are executable only by `service_role` .
- **Push senders** distrust the anon JWT and gate on the service-role bearer or a cron secret with a constant-time compare. Users cannot address pushes to other users' tokens.

- **Password vault cryptography** is correct: AES-256-GCM with library-generated per-message nonces, PBKDF2-HMAC-SHA256 at 210k rounds with a 32-byte per-user salt, a verifier rather than a stored hash, iterations honoured from the record, and a write-once key row. The key never leaves memory in the live code path.
- **Every security-relevant random source** is `Random.secure()`. Share tokens are generated in Postgres, 48-bit for public links and 122-bit for view-once and access keys.
- **No card numbers or CVVs** exist anywhere; the model stores last-four only with a database check constraint.
- **No real secret has ever been committed.** The full git history contains no release keystore, service account, `.env`, or `key.properties`. The client ships a `sb_publishable_key`, not a service-role key. No JWTs are embedded.
- **Sign-in** uses PKCE; an attacker-supplied `#access_token=` or foreign `code` in a deep link is rejected. OTP and TOTP verification are server-side. Demo login is compile-time gated and off by default.
- **Biometric lock** re-prompts on every unlock, stores no bypassable "authenticated" flag, and fails closed. Enabling and disabling both require a successful prompt. Delete account and change password re-authenticate.
- **Sign-out ordering** is correct: push token removed before session teardown, then a broad in-memory and preferences reset.
- **Guest mode** is in-memory only, every write path is gated, and nothing is persisted.
- **UPI payments** allow-list the scheme, rebuild the URI from known keys only so `intent://`, `javascript:` and `file://` can never reach a launcher, and show payee and amount with a first-time consent dialog.
- **Every url_launcher call** is scheme or host allow-listed, and QR-scanned URLs show the full URL in a dialog before opening.
- **Share function HTML** escapes all dynamic text, sets `Cache-Control: no-store`, proxies bytes so bucket paths and signed URLs never reach the client, and re-checks document ownership when serving.
- **Web share site** has no `dangerouslySetInnerHTML`, no secrets in the browser bundle, and no open redirect. Received filenames are sanitised so `../` cannot survive.
- **OCR, document scanning and barcode scanning** run entirely on-device with ML Kit; no image leaves the phone for recognition. The `OcrStage.uploading` label is cosmetic.
- **Exports and backups** contain no passwords, vault key, salt or refresh tokens, and are written to app-private storage then the user's private bucket folder.

- **No analytics, crash, ad or attribution SDK.** No advertising ID, IMEI, location, contacts or SMS permissions. Market and currency feeds are keyless public endpoints that receive no user data.
- **Release build** has R8 minification and resource shrinking, no cleartext traffic, no ATS exceptions, targetSdk 36, and complete iOS usage strings. All exported components are protected by system permissions except the deep-link activity.
- **Sensitive debug dumps** (raw Aadhaar OCR text, full OCR passes) are behind `kDebugMode`. Push tokens are truncated before logging.
- **Per-record save consent** precedes storing documents, cards, investments and passwords, and payment-app launch has a separate revocable consent.

Suspected and ruled out

Items that looked like problems on first pass and were confirmed safe. They should not appear in a fix list.

SUSPECT	WHY IT IS NOT A FINDING
Supabase key and Google OAuth client IDs in <code>supabase_config.dart</code>	Publishable key and client IDs are public by design; RLS and edge-function auth are the real boundary.
Google API keys in <code>google-services.json</code> and <code>firebase_options.dart</code>	Firebase app identifiers, expected in every client. Only FCM is used; no Firestore or Realtime Database rules are in play.
<code>service_role</code> and <code>Bearer</code> matches under <code>supabase/</code>	All are <code>Deno.env.get(...)</code> , SQL grants, or documentation placeholders. No key value exists.
Keystore password <code>"android"</code> in Gradle	Standard debug password. The issue is the committed keystore (H9), not the password.
<code>android:debuggable="true"</code> in the merged manifest	That is the debug variant's output; the release build type never sets it.
Session injection via <code>ino://</code> deep link with <code>#access_token=</code>	PKCE flow only accepts a <code>?code=</code> matched to a local verifier; implicit-grant fragments are rejected.
Biometric result stored as a flippable boolean	Only the preference is stored; unlock state is in-memory and re-prompted every time.
Demo account shared across installs	Button only appears when a <code>--dart-define</code> password is injected at build time; production cannot show it.

XSS in the share function's HTML viewer	All dynamic text passes through <code>escapeHtml</code> , ids through <code>encodeURIComponent</code> , script literals through <code>JSON.stringify</code> .
View-once double-spend race	Single conditional UPDATE with RETURNING; exactly one winner.
Static IV or nonce reuse in the vault	The library generates a fresh 12-byte nonce inside <code>encrypt()</code> ; no nonce argument is ever passed.
Client-side 192-bit share token generator	Dead code; real tokens are generated in Postgres.
Path traversal in the Next.js file route	Token and index are <code>encodeURIComponent</code> -ed before the upstream fetch.
Aadhaar parser printing the full number	Gated by <code>kDebugMode</code> ; not in release.
<code>VaultCrypto.unlockForTest</code> with 1,000 iterations	<code>@visibleForTesting</code> , sets an in-memory key only, cannot affect stored data.
"StorageAnalytics" and "analytics" hits	Internal storage-usage and share-view counters, not a tracking SDK.
CORS <code>*</code> on the share function	Required for an anonymous public viewer; nothing credentialed is exposed by CORS itself.
Non-constant-time compare on view-once <code>access_key</code>	122-bit random value; a timing leak is not practically exploitable.
Release keystore path in <code>PRODUCTION_GOOGLE_SIGNIN.md</code>	Stale documentation; the file is not on disk and was never committed.

Play Console readiness

REQUIREMENT	STATUS	EVIDENCE
Application ID is not <code>com.example.*</code>	FAIL	C4
Public privacy-policy URL	FAIL	C3
Policy names data types, purposes, third parties, retention	FAIL	H8

In-app account deletion actually deletes	FAIL	C2
Web account-deletion request URL	FAIL	C3
Target audience / 18+ declaration	FAIL	M13
Photo and Video permissions policy	FAIL	M9
No misleading security claims	FAIL	L14
Backup rules for an app storing credentials	FAIL	H5
Terms and conditions present	PARTIAL	Present but no age, governing law or entity name
Consent to terms at sign-up	PARTIAL	Email path yes, Google path no (M12)
Permission rationale before OS prompt	PARTIAL	Camera contextual; notifications at launch, mic none
Exact-alarm permission declaration form	NEEDS INFO	Legitimate reminder use; declare in Console
Data Safety: financial info, ID numbers, voice, device IDs	NEEDS INFO	Use the map below
targetSdk 35 or higher	PASS	targetSdk 36, minSdk 24
Data encrypted in transit	PASS	All endpoints HTTPS; no cleartext
No ad, analytics or tracking SDK	PASS	Only firebase_core and firebase_messaging
CVV and full card number never stored	PASS	Model has no such fields
Release signing externalised, keystore not committed	PASS	key.properties ignored, jks never in history
Guest mode leaves no data	PASS	In-memory only

Data Safety form mapping

What the app actually collects, where it goes, and whether the user can remove it today. Fill the Play form from this table, not from the current policy text.

DATA	SENT TO	CLIENT-SIDE ENCRYPTED	DELETABLE IN-APP TODAY
Email, name, phone, Google profile	Supabase Auth and <code>users</code> ; Google Sign-In	No	Profile row yes; auth row no (C2)
Government ID images (Aadhaar, PAN, passport, DL, voter)	Supabase Storage, private bucket	No	Yes, except share copies
ID numbers, DOB, gender, expiry from OCR	Supabase wallet tables	No (H7)	Yes
OCR processing	On-device ML Kit only	n/a	n/a
Cards: bank, holder, last four, expiry	Supabase	No	Yes
Vault passwords	Supabase, AES-GCM ciphertext	Yes (but plaintext local cache, H3)	Rows yes; key record orphaned
Expenses, investments, property, net worth, notes, reminders	Supabase	No	Wallet rows yes; notes, reminders, expenses orphaned
Payment QR (UPI)	Supabase	No	Orphaned
Family-vault membership, shared docs, audit log	Supabase	No	Orphaned
Share links and view analytics	Supabase; Vercel share site	No	Per-share yes; not on account delete
Push token, platform	Supabase <code>device_tokens</code> ; Google FCM	No	On sign-out only
Firebase installation ID	Google Firebase, every install including guests	n/a	No
Notification titles and bodies	Google FCM; <code>push_log</code> , <code>notification_outbox</code> with no retention	No	Orphaned

Voice audio	OS speech recogniser, may be cloud (M11)	n/a	n/a
IP address and user agent	Supabase, FCM, fonts.gstatic.com (runtime font fetch), swissquote.com, frankfurter.dev	n/a	n/a
Offline document copies	Local app directory only	No	Not wiped on sign-out or delete (H2)
Biometrics	OS only; app stores a boolean	n/a	n/a

Recommended fix order

Ordered by risk removed per hour of work. The first block is the minimum before submission; it is roughly two to three days.

Before submitting

- Validate and parameterise the share lookup, add rate limiting**
 C1, M5. One function, under an hour. Deploy immediately even before the rest.
- Revoke the anon-callable RPCs, fix the vault_members INSERT policy**
 H6, M1, L7. A single migration.
- Set allowBackup false, move tokens and password cache to secure storage**
 H3, H4, H5. Also pass a secure localStorage to Supabase.initialize.
- Gate the splash path on MFA; purge offline data on sign-out and keep the lock on**
 H1, H2, M7, M8.
- Write and commit a real delete_account function; fail loudly if it is missing**
 C2.

- 6 **Rename the application ID and rebuild the Firebase, OAuth and asset-link bindings**
C4, L12. Do this before any Play upload; it is immutable afterwards.

- 7 **Rewrite the privacy policy and terms, host them, add the deletion page and the 18+ clause**
C3, H8, M13, L14. Use the data-safety map as the source of truth and remove the export claim or wire the export.

- 8 **Remove the media permissions and fix the CI signing path**
M9, H9.

- 9 **Confirm the documents-bucket folder policy in the dashboard and commit it**
M17. Ten minutes, and it is the one item that could be a critical if wrong.

First week after

- 10 **Mask and encrypt ID numbers**
H7. Reuse the vault key pattern.

- 11 **Share hardening: MIME allow-list, nosniff and CSP, password in POST body with a salted hash**
M3, M4, L9, L10.

- 12 **Consent and notifications: defer the push prompt, honour the toggle, private lock-screen visibility, log retention**
M10, M12.

- 13 **FLAG_SECURE on document, vault and card screens; clipboard auto-clear; drop the vault key on background**
L1, M14, L15.

- 14 **Server-side quota, uniform invite responses, wallets view without created_by**
M6, M2, L5.

- 15 **Delete the dead vault stack and legacy SQL files; fix the failing isolation tests; bundle fonts**
I1, L8, I6, and the runtime Google Fonts fetch.

After the "before submitting" block, the projected score is roughly 7 out of 10, with the backend at 9, auth at 8, and privacy at 6. The first-week block would bring the app to a position that is defensible under both Play review and India's DPDP Act.

Method: six parallel source audits (auth and session, cryptography and local data, Supabase schema and functions, sharing and web, platform and build, privacy and policy), each required to list what it verified as sound and what it ruled out. Every Critical and High finding was then re-read in the code by a second pass before inclusion. The 13 security-related test suites were run: 138 pass, 3 fail on a harness issue (I6). No files were modified.